

Reviewer Pack — Minimal Deligne-Lusztig Tree Depth and Communication Complex...

Entry 868e9570be1c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 14:15:53 UTC

Minimal Deligne-Lusztig Tree Depth and Communication Complexity Rank Variance Correlation

Entry ID: 868e9570be1c **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 14:15:53 UTC

1. Conjecture

Field A (mathematical branch): Geometric Representation Theory (Deligne-Lusztig Trees) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every CNF φ with m clauses, the minimal depth of a Deligne-Lusztig tree associated with the incidence algebra of φ is linearly correlated with its communication complexity rank variance, such that this correlation coefficient exceeds 0.95 for instances with $n \leq 40$ variables.

Rationale (proposer's reasoning):

Geometric Representation Theory provides tools to study algebraic structures in terms of geometric objects, while Deligne-Lusztig trees are known to capture the structure of representations of reductive groups. Communication complexity rank variance measures the complexity of a function when computed by two parties with limited communication. The conjecture suggests that these geometric structures could provide insights into the complexities of boolean functions.

Taxonomy category: geometric_representation_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8583d0be295d5999

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean correlation coefficient between minimal Deligne-Lusztig tree depth and communication complexity rank variance exceeds 0.95 for instances with $n \leq 40$ variables, across all seeds. The criterion is falsified if any seed yields a mean correlation coefficient below 0.95.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (Deligne-Lusztig Trees) AND (Communication Complexity rank variance) - (Incidence algebra) AND (CNF depth correlation) - (Matrix rank variance) AND (Geometric representation theory)

Top relevant hits considered: - [<http://arxiv.org/abs/1204.1444v2>] Computing the minimum rank of a loop directed tree - [<http://arxiv.org/abs/2105.01963v3>] One-way communication complexity and non-adaptive decision trees - [<http://arxiv.org/abs/2409.16853v3>] Periods of Deligne-Lusztig Characters associated to Spherical Varieties - [<http://arxiv.org/abs/1306.0725v5>] Algebraic quotient modules and subgroup depth - [<http://arxiv.org/abs/0707.3756v1>] Finite depth and Jacobson-Bourbaki correspondence - [<http://arxiv.org/abs/0812.3044v3>] Exploration of finite dimensional Kac algebras and lattices of intermediate subfactors of irreducible inclusions - [<http://arxiv.org/abs/1005.0979v1>] Supersymmetry in Random Matrix Theory - [<http://arxiv.org/abs/math/0402188v5>] Structures and Representations of Generalized Path Algebras

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

# Helper functions for linear algebra and group operations
def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        # Find pivot
```

```

max_row = i
for j in range(i+1, n):
    if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
        max_row = j
augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]

# Eliminate below pivot
for j in range(i+1, n):
    factor = augmented_matrix[j][i] / augmented_matrix[i][i]
    for k in range(n + 1):
        augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

# Back substitution
x = [0] * n
for i in range(n-1, -1, -1):
    x[i] = augmented_matrix[i][-1]
    for j in range(i+1, n):
        x[i] -= augmented_matrix[i][j] * x[j]
    x[i] /= augmented_matrix[i][i]

return x

def matrix_inverse(A):
    n = len(A)
    I = [[Fraction(0) if i != j else Fraction(1) for j in range(n)] for i in range(n)]
    A_augmented = [A[i] + I[i] for i in range(n)]

    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A_augmented[j][i]) > abs(A_augmented[max_row][i]):
                max_row = j
        A_augmented[i], A_augmented[max_row] = A_augmented[max_row], A_augmented[i]

        # Eliminate below pivot
        for j in range(i+1, n):
            factor = A_augmented[j][i] / A_augmented[i][i]
            for k in range(2 * n):
                A_augmented[j][k] -= factor * A_augmented[i][k]

    # Back substitution
    for i in range(n):
        x_i = Fraction(A_augmented[i][-1])
        for j in range(i+1, n):
            x_i -= A_augmented[i][j] * Fraction(A_augmented[j][-1])
        x_i /= A_augmented[i][i]

    # Normalize row
    for k in range(2 * n):
        A_augmented[i][k] *= x_i

    return [row[n:] for row in A_augmented]

# Function to generate a random CNF with n variables and m clauses
def generate_cnf(n, m):

```

```

cnf = []
for _ in range(m):
    clause = set()
    while len(clause) < 3:
        var = random.randint(1, n)
        sign = random.choice([True, False])
        if (var, not sign) not in clause and (-var, sign) not in clause:
            clause.add((var, sign))
    cnf.append(clause)
return cnf

# Function to compute the incidence algebra of a CNF
def incidence_algebra(cnf):
    n = max(abs(var) for var, _ in cnf)
    algebra = [[0] * (n + 1) for _ in range(n + 1)]

    for clause in cnf:
        for i in range(1, n + 1):
            if (i, True) not in clause and (-i, False) not in clause:
                for j in range(i + 1, n + 1):
                    if (j, True) not in clause and (-j, False) not in clause:
                        algebra[i][j] += 1

    return algebra

# Function to compute the minimal depth of a Deligne-Lusztig tree
def deligne_lusztig_tree_depth(algebra):
    n = len(algebra)
    adjacency_matrix = [[0] * (n + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for j in range(i + 1, n + 1):
            if algebra[i][j] > 0:
                adjacency_matrix[i][j] = 1
                adjacency_matrix[j][i] = 1

    # Find the maximum independent set
    max_independent_set = []
    visited = [False] * (n + 1)

    def dfs(node):
        if visited[node]:
            return False
        visited[node] = True
        max_independent_set.append(node)
        for neighbor in range(1, n + 1):
            if adjacency_matrix[node][neighbor] == 1:
                if not dfs(neighbor):
                    return False
        return True

    for node in range(1, n + 1):
        if not visited[node]:
            dfs(node)

    # The depth is the size of the maximum independent set minus one
    return len(max_independent_set) - 1

```

```

# Function to compute the communication complexity rank variance
def communication_complexity_rank_variance(cnf):
    n = max(abs(var) for var, _ in cnf)
    incidence_matrix = [[0] * (n + 1) for _ in range(n + 1)]

    for clause in cnf:
        for i in range(1, n + 1):
            if (i, True) not in clause and (-i, False) not in clause:
                for j in range(i + 1, n + 1):
                    if (j, True) not in clause and (-j, False) not in clause:
                        incidence_matrix[i][j] += 1

    # Compute the rank of the incidence matrix
    rank = len(gaussian_elimination(incidence_matrix, [0] * (n + 1)))

    # The variance is the difference between the maximum and minimum ranks
    return n - rank

# Function to run a single trial for a given seed
def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for _ in range(30):
        n = random.randint(5, 40)
        m = random.randint(n, n * (n - 1))
        cnf = generate_cnf(n, m)

        algebra = incidence_algebra(cnf)
        depth = deligne_lusztig_tree_depth(algebra)
        variance = communication_complexity_rank_variance(cnf)

        results.append((depth, variance))

    if not results:
        return {
            "metric_name": "correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_results"
        }

    mean_depth = sum(depth for depth, _ in results) / len(results)
    mean_variance = sum(variance for _, variance in results) / len(results)
    correlation_coefficient = sum((depth - mean_depth) * (variance - mean_variance) for depth, variance in results) / len(results)

    return {
        "metric_name": "correlation",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for n, _ in results),
        "conjecture_holds": correlation_coefficient > 0.95,
        "counterexample": ""
    }

```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(2, 31)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"low_correlation\" first_failing_seed={first_fai

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55a05dc7.py", line 209, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55a05dc7.py", line 174, in run_trial
    algebra = incidence_algebra(cnf)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55a05dc7.py", line 102, in incidence_algebra
    n = max(abs(var) for var, _ in cnf)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55a05dc7.py", line 102, in <genexpr>
    n = max(abs(var) for var, _ in cnf)
          ~~~~~^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Debug the test code to ensure it runs successfully and produces the required data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14546
2	propose	ollama_remote	glm4:latest	0	0	12386
3	propose	ollama_remote	glm4:latest	0	0	13351
4	preregistration	ollama_remote	glm4:latest	0	0	15810
5	novelty	ollama_remote	glm4:latest	0	0	9939
6	novelty	ollama_remote	glm4:latest	0	0	9773
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22265
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16855
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15210
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24655
11	judge	ollama_remote	glm4:latest	0	0	17465

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 172256 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/868e9570be1c.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/868e9570be1c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/868e9570be1c.tar.gz (if generated)